Assignment_5_cwood19

Chris Wood

2026-06-18

Load libraries and datasets. The dataset Cereals.csv includes nutritional information, store display, and consumer ratings for 77 breakfast cereals. The following fields appear in the set:
1. name (datatype: chr)
2. mfr (datatype: chr)
3. type (datatype: chr)
4. calories (datatype: int)
5. protein (datatype: int)
6. fat (datatype: int)
7. sodium (datatype: int)
8. fiber (datatype: num)
9. carbo (datatype: num)
10. sugars (datatype: int)
11. potass (datatype: int)
12. vitamins (datatype: int)
13. shelf (datatype: int)
14. weight (datatype: num)
15. cups (datatype: num)
16. rating (datatype: num)
Data Preprocessing: Remove all cereals with missing values.

```
# Load packages
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
library(cluster)
```

```
## Warning: package 'cluster' was built under R version 4.5.3
```

```
library(mclust)
```

```
## Warning: package 'mclust' was built under R version 4.5.3
```

```
## Package 'mclust' version 6.1.2
## Type 'citation("mclust")' for citing this R package in publications.
```

```r
# Load dataset
cereals <- read.csv("Cereals.csv")

# headers & data type
# names(cereals)
# str(cereals)

# Original rows
# nrow(read.csv("Cereals.csv"))

# Omit incomplete rows
cereals <- cereals[complete.cases(cereals), ]

# Number removed rows
# nrow(read.csv("Cereals.csv")) - nrow(cereals)

# Select numeric variables for clustering
cereal_num <- cereals[, c("calories", "protein", "fat", "sodium",
                          "fiber", "carbo", "sugars", "potass",
                          "vitamins", "shelf", "weight", "cups")]

# Normalize variables
cereal_scaled <- scale(cereal_num)
```

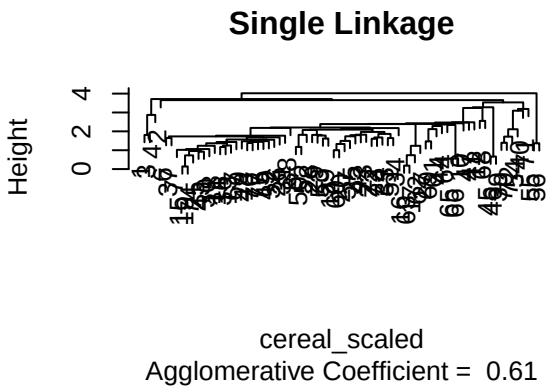
Apply hierarchical clustering to the data using Euclidean distance to the normalized measurements. Use Agnes to compare the clustering from single linkage, complete linkage, average linkage, and Ward. Choose the best method.

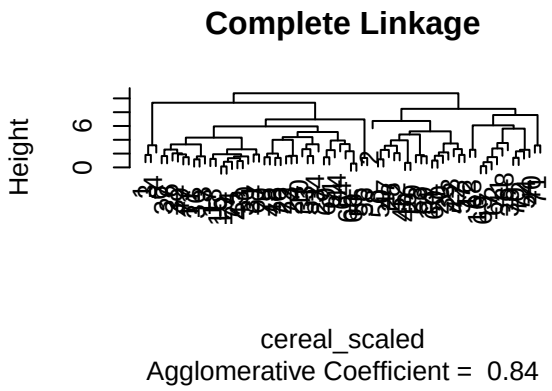
```r
# Single linkage
agnes_single <- agnes(cereal_scaled,
                      metric = "euclidean",
                      method = "single")

# Complete linkage
agnes_complete <- agnes(cereal_scaled,
                        metric = "euclidean",
                        method = "complete")

# Average linkage
agnes_average <- agnes(cereal_scaled,
                       metric = "euclidean",
                       method = "average")

# Ward linkage
agnes_ward <- agnes(cereal_scaled,
                    metric = "euclidean",
                    method = "ward")

data.frame(
  Method = c("Single", "Complete", "Average", "Ward"),
  AC = c(
    agnes_single$ac,
    agnes_complete$ac,
```

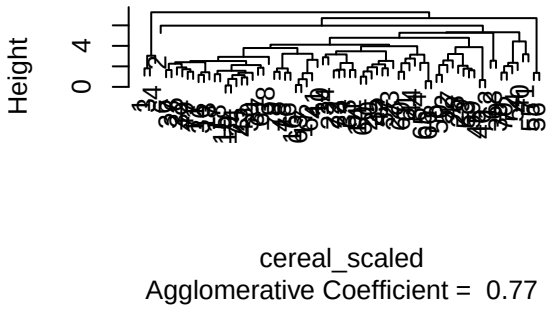
```
    agnes_average$ac,
    agnes_ward$ac
  )
)
```

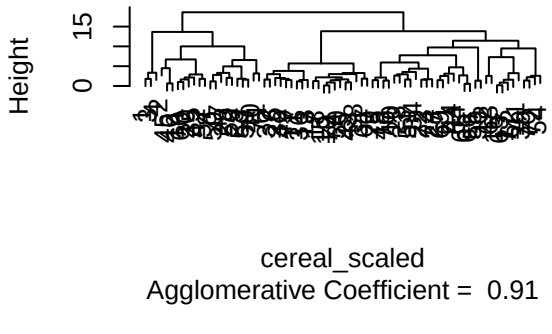
```
##     Method        AC
## 1   Single 0.6145986
## 2 Complete 0.8377131
## 3  Average 0.7689075
## 4     Ward 0.9075685
```

```
par(mfrow = c(2,2))

# View data trees
plot(agnes_single, which.plots = 2, main = "Single Linkage")
plot(agnes_complete, which.plots = 2, main = "Complete Linkage")
plot(agnes_average, which.plots = 2, main = "Average Linkage")
plot(agnes_ward, which.plots = 2, main = "Ward Linkage")
```










```
# Agglomerative Coefficients
ac_values <- c(
  Single = agnes_single$ac,
  Complete = agnes_complete$ac,
  Average = agnes_average$ac,
```

```
  Ward = agnes_ward$ac
)

# Choose best AC
ac_values
```

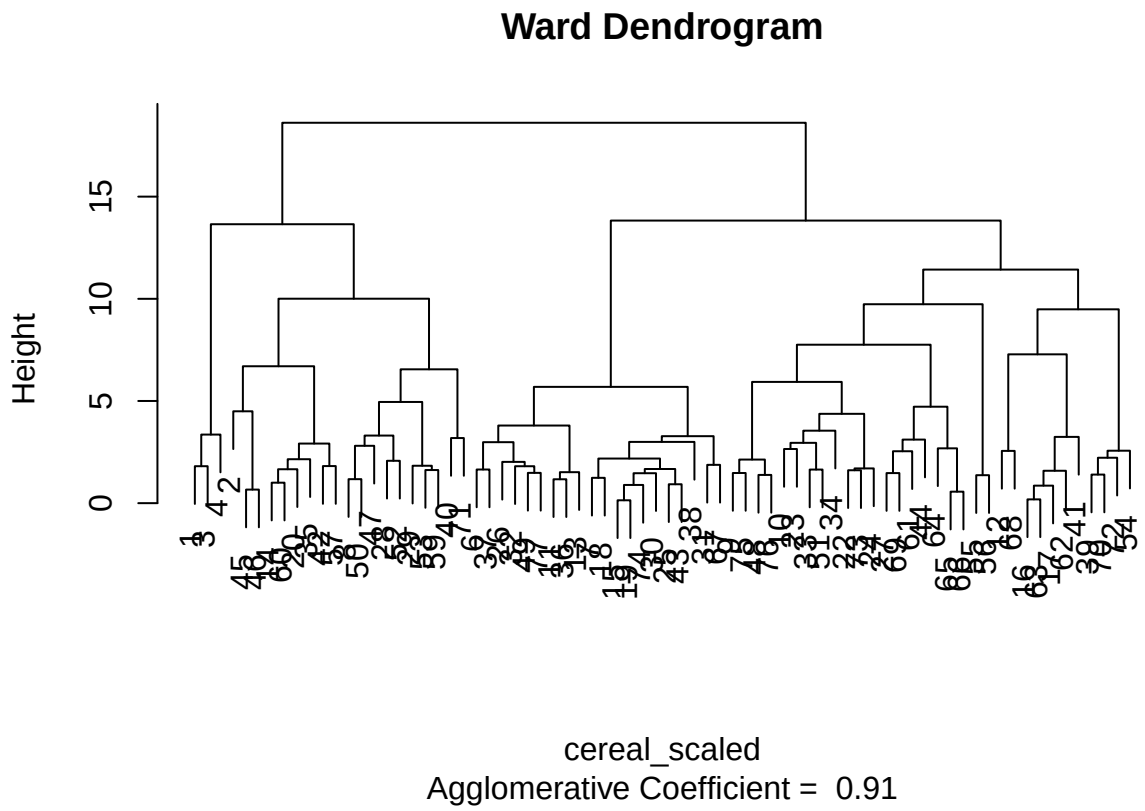
```
##    Single  Complete   Average      Ward
## 0.6145986 0.8377131 0.7689075 0.9075685
```

```
which.max(ac_values)
```

```
## Ward
##    4
```

Based on the data clusters, the Ward Linkage AC is significantly higher than all others, showing optimal balanced approach.
How many clusters would you choose?

```
# Plot dendrogram
plot(agnes_ward, which.plots = 2, main = "Ward Dendrogram")
```




```
# Find average silhouette width
hc <- as.hclust(agnes_ward)
```

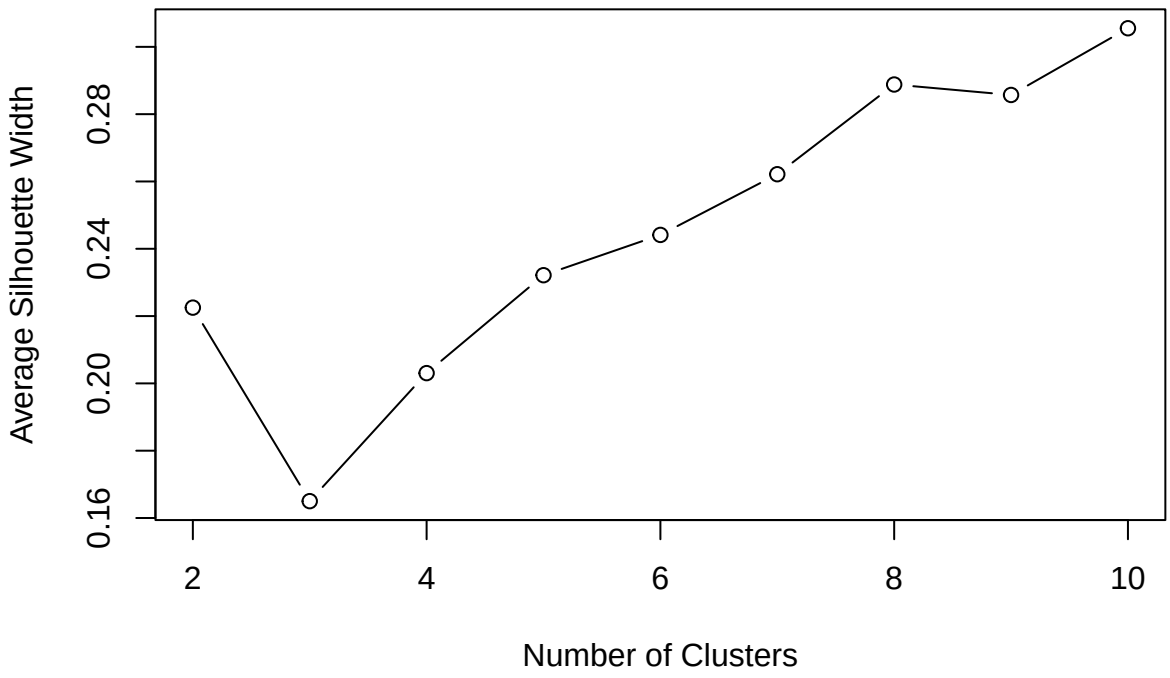
```r
sil_width <- numeric(10)

for (k in 2:10) {
  clusters <- cutree(hc, k = k)
  sil <- silhouette(clusters, dist(cereal_scaled))
  sil_width[k] <- mean(sil[, 3])
}

sil_width[2:10]
```

```
## [1] 0.2225119 0.1650075 0.2030525 0.2321482 0.2441210 0.2621551 0.2888159
## [8] 0.2857053 0.3055297
```

```r
# Find best K
which.max(sil_width)
```

```
## [1] 10
```

```r
# Plot results
plot(2:10, sil_width[2:10],
     type = "b",
     xlab = "Number of Clusters",
     ylab = "Average Silhouette Width")
```



Comparing the dendrogram with the silhouette results, checking from 2:10, setting K=10 has an optimal width of .3055.
Comment on the structure of the clusters and on their stability.
Hint: To check stability, partition the data and see how well clusters formed based on one part apply to the other part. - Cluster partition A - Use the cluster centroids from A to assign each record in partition B (each record is assigned to the cluster with the closest centroid). - Assess how consistent the cluster assignments are compared to the assignments based on all the data.

```
set.seed(123)

# Create a 50/50 split
idx <- sample(1:nrow(cereal_scaled),
              size = floor(0.5 * nrow(cereal_scaled)))

A <- cereal_scaled[idx, ]
B <- cereal_scaled[-idx, ]

# Cluster partition A
agnes_A <- agnes(A,
                 metric = "euclidean",
                 method = "ward")

hc_A <- as.hclust(agnes_A)

clusters_A <- cutree(hc_A, k = 10)

# Compute centroids for A
centroids <- aggregate(A,
                       by = list(cluster = clusters_A),
                       FUN = mean)

centroids
```

```
##    cluster   calories    protein        fat      sodium      fiber       carbo
## 1        1 -0.2701265  0.6071316  0.1655367 -0.07891203  0.4433144 -0.38022956
## 2        2  0.8217292  0.4522084  0.3310734  0.43456036  0.7871852 -0.25175016
## 3        3  0.2002114 -1.1280086  0.0000000 -0.14334778 -0.8152525 -0.59864453
## 4        4  0.1498180 -0.4773310  0.0000000  0.45469653 -0.8977815  1.61120105
## 5        5 -0.6060820  0.6845932 -0.7449152 -1.96164410  0.1338308  0.64760560
## 6        6  2.1655516  1.3817478  1.9864405 -0.48163546  0.3401532  0.32640711
## 7        7 -0.3541153  0.1423619  0.0000000  0.77687528  0.2713791 -0.05903107
## 8        8  0.1498180  3.2408266 -0.9932203  0.81714763 -0.4851366  0.32640711
## 9        9  0.1498180 -0.4773310 -0.3310734  1.21987107 -0.8977815  1.69685398
## 10      10  1.6616182  0.4522084  0.0000000  0.33387950  0.7527981  0.06944832
##        sugars     potass   vitamins      shelf     weight       cups
## 1  -0.3306732  0.4324735 -0.1818422  0.9419715 -0.2008324 -1.1523290
## 2   0.8928177  1.2202045 -0.1818422  0.7416672  1.7763687 -0.6503111
## 3   1.1451628 -0.8749248 -0.1818422 -0.5002193 -0.2008324  0.5276635
## 4  -0.9424187 -0.8960877  3.1822385  0.9419715 -0.2008324  0.7567534
## 5  -1.0571210  0.2149657 -0.7425223 -0.8607671 -0.2008324 -0.1553638
## 6   0.8928177  1.0085753 -0.1818422  0.9419715 -0.2008324  0.7567534
## 7  -0.1777369  0.1620584 -0.1818422 -1.4616799 -0.2008324 -0.5301096
## 8  -0.9424187 -0.6139154 -0.1818422 -1.4616799 -0.2008324  0.7567534
## 9  -0.9424187 -0.8020303 -0.1818422 -0.2598542 -0.2008324  1.1102872
```

```
## 10  1.5810314  1.8550922  3.1822385  0.9419715  3.0582904  0.7567534
```

```r
# Remove the cluster label
centroid_matrix <- as.matrix(centroids[, -1])

# Assign records in B to centroid
assign_cluster <- function(x, centroids) {
  d <- apply(centroids, 1,
             function(c) sqrt(sum((x - c)^2)))
  which.min(d)
}

clusters_B_from_A <- apply(B, 1,
                           assign_cluster,
                           centroids = centroid_matrix)

# Compare the clustering
agnes_full <- agnes(cereal_scaled,
                    metric = "euclidean",
                    method = "ward")

hc_full <- as.hclust(agnes_full)

clusters_full <- cutree(hc_full, k = 10)

clusters_B_full <- clusters_full[-idx]

# Cluster stability
adjustedRandIndex(clusters_B_from_A,
                  clusters_B_full)
```

```
## [1] 0.5097763
```

Viewing 10 clusters with an average silhouette width of .306 indicates toward moderate separation between clusters and an index of .510 leads toward moderate stability and reproducibility. Some boundaries between clusters are not very sharp and contains some small overlapping among clusters, but overall groupings are meaningful.
The elementary public schools would like to choose a set of cereals to include in their daily cafeterias. Every day a different cereal is offered, but all cereals should support a healthy diet. For this goal, you are requested to find a cluster of "healthy cereals."
Should the data be normalized? If not, how should they be used in the cluster analysis?

```r
# Create a health score
# Based on low sugar, sodium, calories
# Based on high fiber, protein
centroids <- aggregate(cereal_scaled,
                       by = list(cluster = clusters_full),
                       mean)

centroids$health_score <-
  (-centroids$sugars) +
  (-centroids$sodium) +
  (-centroids$calories) +
```

```
  centroids$fiber +
  centroids$protein

# Rank clusters best to worst
centroids[order(centroids$health_score, decreasing = TRUE), ]
```

```
##    cluster    calories     protein        fat      sodium      fiber      carbo
## 1        1 -2.20187108  1.38174776 -0.3310734  0.17279012  3.6413124 -2.0718749
## 10      10 -2.87378226 -0.94210075 -0.9932203 -1.96164410 -0.6914590 -0.8299074
## 8        8 -0.78605825  0.18662567 -0.8513317 -1.93575474  0.1633054  0.4365323
## 6        6  0.14981803  3.24082658  0.0000000  1.17959872 -0.2788141  0.4548865
## 2        2  0.54176622  0.86533698  1.6553671 -0.56553618  0.1796802 -0.5729487
## 5        5 -0.35411535 -0.01256134 -0.4138418  0.39428802  0.1338308  0.4548865
## 9        9 -0.10214866 -0.01256134 -0.2483051  0.72653485 -0.3819753  0.9688041
## 4        4  1.25847147  0.45220836  0.3972881  0.35804291  0.6083724  0.2107757
## 7        7  0.04903136 -0.66323893 -0.7945762  1.32457916 -0.8152525  1.8167681
## 3        3  0.22540804 -0.94210075  0.0000000  0.09224544 -0.6914590 -0.5729487
##        sugars       potass   vitamins      shelf     weight        cups
## 1  -0.7894824  2.983781331 -0.1818422  0.9419715 -0.2008324 -1.84525525
## 10 -1.6306324 -0.931359220 -1.3032024  0.9419715 -3.4599552  0.75675340
## 8  -0.9424187  0.121748084 -0.6624252 -0.6032330 -0.3591327 -0.06748537
## 6  -1.1718233 -0.261200027 -0.1818422 -1.4616799 -0.2008324  1.28705407
## 2   0.1026465  0.467745061 -0.3064378  0.8084353 -0.2008324 -0.65738174
## 5  -0.5983119  0.003336497 -0.1818422  0.1407544 -0.2008324 -0.16596978
## 9  -0.7703653 -0.508100782  3.1822385  0.9419715 -0.2008324  0.75675340
## 4   0.8698773  1.043846823  0.4909739  0.8217889  2.0479623 -0.47354417
## 7  -1.0341806 -0.924304913 -0.1818422 -1.2213148 -0.2008324  1.29129648
## 3   1.0189902 -0.772637306 -0.1818422 -0.6204019 -0.2008324  0.25402836
##    health_score
## 1     7.8416234
## 10    4.8324990
## 8     4.0141628
## 6     2.8044190
## 2     0.9661407
## 5     0.6794087
## 9    -0.2485576
## 4    -1.4258109
## 7    -1.8179214
## 3    -2.9702035
```

```
# Look for good diversity (5-8)
table(clusters_full)
```

```
## clusters_full
##  1  2  3  4  5  6  7  8  9 10
##  3  9 20 10 12  2  5  7  4  2
```

```
# View cereal choices
# healthy_cluster_id <- centroids$cluster[which.max(centroids$health_score)]
# Force cluster 8 as chosen cluster
healthy_cluster_id <- 8
subset(cereals, clusters_full == healthy_cluster_id)
```

```
##                         name mfr type calories protein fat sodium fiber carbo
## 27       Frosted_Mini-Wheats   K    C      100       3   0      0     3    14
## 44                     Maypo   A    H      100       4   1      0     0    16
## 61            Raisin_Squares   K    C       90       2   0      0     2    15
## 64            Shredded_Wheat   N    C       80       2   0      0     3    16
## 65    Shredded_Wheat_'n'Bran   N    C       90       3   0      0     4    19
## 66 Shredded_Wheat_spoon_size   N    C       90       3   0      0     3    20
## 69   Strawberry_Fruit_Wheats   N    C       90       2   0     15     3    15
##    sugars potass vitamins shelf weight cups   rating
## 27      7    100       25     2   1.00 0.80 58.34514
## 44      3     95       25     2   1.00 1.00 54.85092
## 61      6    110       25     3   1.00 0.50 55.33314
## 64      0     95        0     1   0.83 1.00 68.23588
## 65      0    140        0     1   1.00 0.67 74.47295
## 66      0    120        0     1   1.00 0.67 72.80179
## 69      5     90       25     2   1.00 1.00 59.36399
```

Aiming for a relatively balanced diet, restrictions are placed to choose options that have lower sodium, calories, and sugar while focusing on higher fiber and protein to support fullness, muscle growth, and avoid empty calories. Although cluster 1 and 10 are listed as extreme benefits and choice, cluster 8 is likely the best all-around alternative since it has seven cereals in its cluster and still generally fits the criteria with balanced options compared to the others with very minimal option but high benefit.